

Informatik Projekt



Fabian Müller, Luca Nettel

Teamname

- Nachdem wir alle unsere Gruppen hatten und voller Tatendrang an unseren Projekten gearbeitet haben, kam leider die schlechte Nachricht, dass mich mein Teampartner Noah verlässt.
- Nach den Wiederholungsklausuren erhielt Fabian dann ebenfalls die Nachricht, dass ihn Luca verlassen wird.
- Am Montagmittag kam uns schließlich die Idee, dass wir das Projekt gemeinsam fertigstellen könnten, damit wir beide noch die Möglichkeit haben, es rechtzeitig abzuschließen.
- Im Laufe der nächsten zwei Tage entstand dann der glorreiche Name:

„Aus 4 macht 2 Raspberry Pi“

Teamvorstellung

- Fabian Müller
 - Programmierung des Roboters
 - Volles Verständnis über Python und GitHub
 - Gestaltung und Design der PowerPoint
- Luca Nettel
 - Erstellung der PowerPoint
 - Gestaltung und Design der PowerPoint
 - Programmierung des Roboters

Beispiel von Cleancode

- **Vermeidung von mehrfach verwendeten Festwerten (Magic Numbers)**
- Zentrale Definition häufig verwendeter Geschwindigkeiten
- Änderungen müssen nur an einer Stelle vorgenommen werden
- Bessere Lesbarkeit durch sprechende Variablennamen (`main_speed`, `inner_turn_speed`, `outer_turn_speed`)

Beispiel von Cleancode

Auslagerung von Konfigurationswerten in config.json

- Zentrale Verwaltung aller veränderbaren Parameter
- Änderungen am Fahrverhalten ohne Anpassung des Quellcodes möglich
- Bessere Wartbarkeit und Übersichtlichkeit
- Vermeidung von fest im Code hinterlegten Werten
- Wiederverwendbarkeit des Programms mit unterschiedlichen Einstellungen auch von anderen Personen

```
slight_outer_turn_speed = config["slight_outer_turn_speed"]  
slight_inner_turn_speed = config["slight_inner_turn_speed"]
```

```
def slight_turn_right():  
    rear_left(slight_outer_turn_speed)  
    front_left(slight_outer_turn_speed)  
    rear_right(slight_inner_turn_speed)  
    front_right(slight_inner_turn_speed)
```

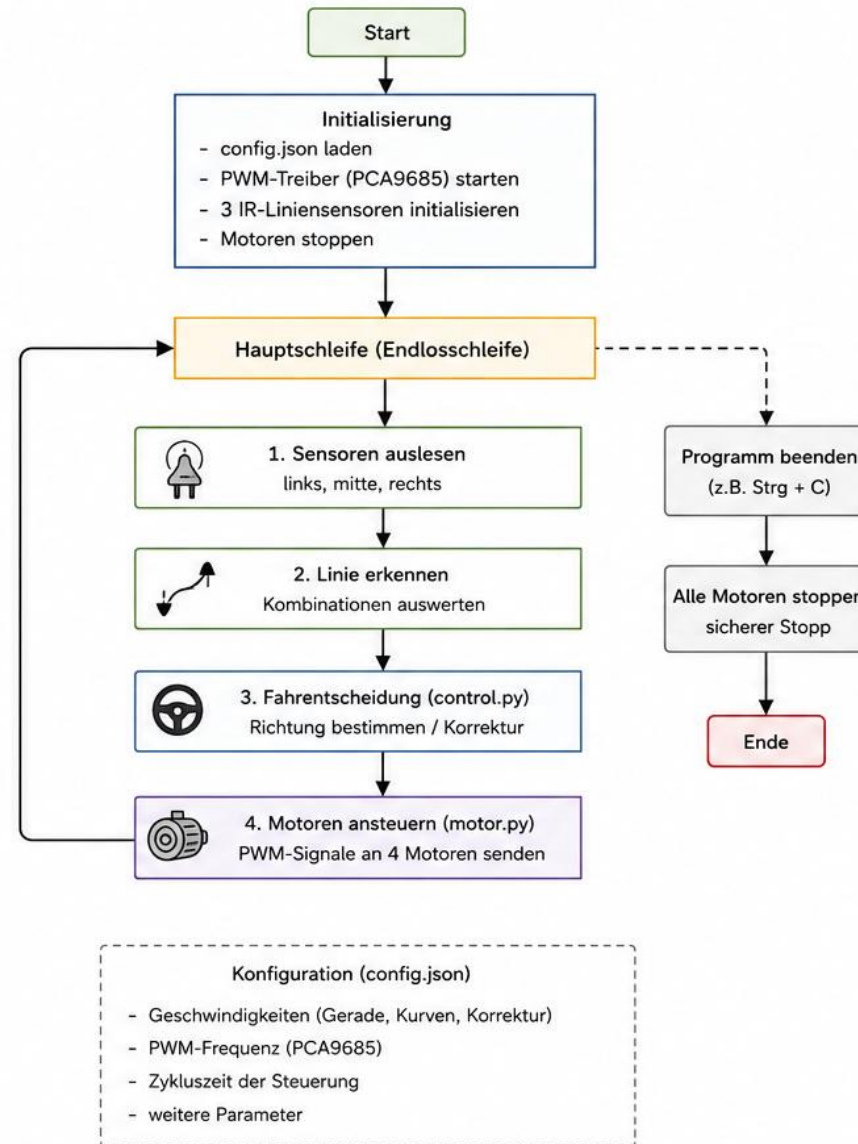
Beispiel von Cleancode

- Saubere Funktionen statt direktem PWM-Code überall:
- `forward()`
- `turn_left()`
- `turn_right()`
- `stop_all()`
- Motorsteuerung über klare Funktionen gekapselt
- Keine direkte PCA9685-Ansteuerung in `control.py` sondern lesbare Befehle

```
elif sensor.detected_middle():  
    motor.forward()  
    last_sensor = "middle"  
  
elif sensor.detected_left():  
    motor.turn_left()  
    last_sensor = "left"  
  
elif sensor.detected_right():  
    motor.turn_right()  
    last_sensor = "right"
```

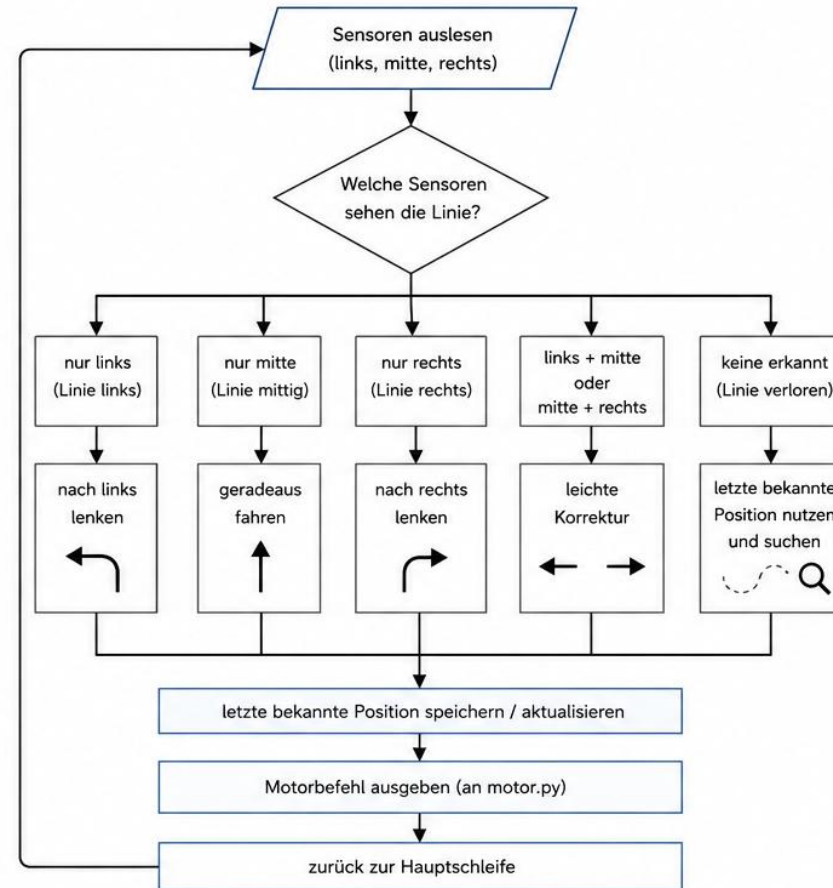
Flussdiagramm der Funktionen

Überblick – Linienfolger Roboter



Flussdiagramm der Funktionen

Ablauf der Fahrentscheidung (control.py)



Module im Überblick

- sensor.py -> liest Sensoren und liefert Werte / Kombinationen
- control.py -> enthält die Fahrlogik und Entscheidungen
- motor.py -> steuert die 4 Motoren über PCA9685
- main.py -> startet Programm und sorgt für sicheres Beenden

Algorithmus erklärung

- Sensor liest den Untergrund
- Prüfe:
 - Ist die Linie unter dem Sensor? → Ja → Aktion: *Linie erkannt* → Nein → Aktion: *Linie nicht erkannt*
- Führe die passende Funktion aus
- Warte auf das nächste Sensor-Ereignis
- Wiederhole
- Das ist der Kern jedes Linienverfolgungs-Systems – egal ob mit einem oder mehreren Sensoren.

Vielen Dank für eure
Aufmerksamkeit